



deti

universidade de aveiro
departamento de eletrónica,
telecomunicações e informática

Base de Dados

Ano letivo 2025/2026

Docente José Manuel Matos Moreira (jose.moreira@ua.pt)

Turma P1, Grupo 2

Sistema de Gestão de Explicações Online - Relatório de Trabalho
Prático final

Fernando Ferreira (faf2@ua.pt) - 119758

Martim Gomes (martimperalta7@ua.pt) - 119488

1. Resumo

O seguinte relatório explica e descreve como foi desenvolvido um sistema de gestão de explicações online. Este projeto engloba não só a implementação de uma base de dados relacional no SQL server, como também uma aplicação gráfica em C# (Windows Forms) que utiliza a base de dados de forma direta.

Após a escolha e levantamento de alguns requisitos principais, desenvolvemos tanto o **der** como o **er** conforme as ideias iniciais que íamos tendo, tendo ainda no início também a questão que foi colocada pelo professor se poderiam ser explicações individuais ou coletivas. Nós optamos por escolher explicações online individuais. Assim sendo, o nosso projeto foi desenhado para gerir de forma eficiente a interação entre os formadores (responsáveis pela criação de cursos, aulas e os respetivos recursos) e os alunos (inscrevem-se em cursos, realizam pagamentos e podem submeter avaliações ao curso que frequentaram). Existem, no entanto, algumas nuances que vão ser referidas no decorrer deste relatório.

NOTA: De forma a testar a nossa aplicação, o novo utilizador deve alterar a variável `connectionString` que se encontra declarada de forma estática no início do código-fonte de cada um dos ficheiros de formulário. Basta substituir os parâmetros `Server`, `Database`, `User Id` e `Password` pelos do novo utilizador.

2. Análise de Requisitos

O sistema foi desenvolvido com base em alguns **requisitos principais**.

Requisitos **funcionais**:

- O sistema deve permitir o registo de utilizadores com nome, email, senha, data de registo e data de nascimento do utilizador.
- O sistema deve distinguir entre alunos e formadores, armazenando informações específicas para cada um deles (biografia e especialidade para os formadores, idade e data de nascimento para os alunos).
- O sistema deve permitir o registo de instituições de ensino (escolas secundárias, politécnicos e universidades) às quais os alunos pertencem.
- Os formadores devem poder criar cursos definindo título, descrição, dificuldade e código de curso.

- Cada curso deve permitir a criação de múltiplas aulas organizadas por número, título, duração e id da aula.
- O sistema deve permitir associar recursos às aulas, registando o nome do arquivo, tipo, tamanho do ficheiro e o id do recurso.
- O sistema deve permitir que um aluno se inscreva em vários cursos.
- O sistema deve processar e registar os pagamentos das inscrições, incluindo o método, id do pagamento, valor, data e estado da transação em questão na altura(pendente/concluído).
- Os alunos podem submeter avaliações sobre os cursos que eles frequentam.
- O sistema deve registar e gerir o enquadramento académico do Aluno, associando esse mesmo aluno a uma instituição de ensino específica (escola secundária, universidade ou politécnico).

Requisitos **não funcionais**:

- As senhas de todos os utilizadores devem ser armazenadas na base de dados de forma encriptada.
- A base de dados deve garantir certas restrições de consistência, como por exemplo, assegurar que a nota de uma Avaliação seja obrigatoriamente um valor entre 1 e 5.
- A base de dados deve suportar o crescimento do número de alunos e o volume de recursos que estão anexados às aulas.
- Todos os pagamentos devem ser registados de forma a evitar perda de dados em caso de falha.
- O sistema de base de dados deve garantir alta disponibilidade, com backups regulares para evitar perda de informação também sobre inscrições.
- A base de dados deve conseguir suportar várias pesquisas ao mesmo tempo (ex: vários alunos a pesquisar cursos ao mesmo tempo) com tempos de resposta inferiores a x segundos, sendo x 2 ou 3 segundos.
- O sistema deve proteger os dados pessoais dos utilizadores de forma extremamente segura.

3. Modelo Entidade-Relacionamento (DER) e Esquema Relacional (ER)

O **DER** representa as entidades, atributos e relações do sistema. Destacam-se duas hierarquias de generalização/especialização (IS-A): Utilizador -> Aluno / Formador e Instituição -> Escola Secundária / Universidade / Politécnico. O atributo idade do aluno é derivado (representado a tracejado), sendo assim calculado a partir da data de nascimento e assim não foi incluído no DDL.

O **ER** resulta da conversão do DER, com as tabelas organizadas no schema Explicacao_Online. As **FKs** (Foreign Keys) das tabelas Inscrever, Avaliacao e Pagamento foram depois alteradas numa fase posterior para referenciar Utilizador em vez de Aluno, de modo a que os formadores também conseguissem inscrever em cursos.

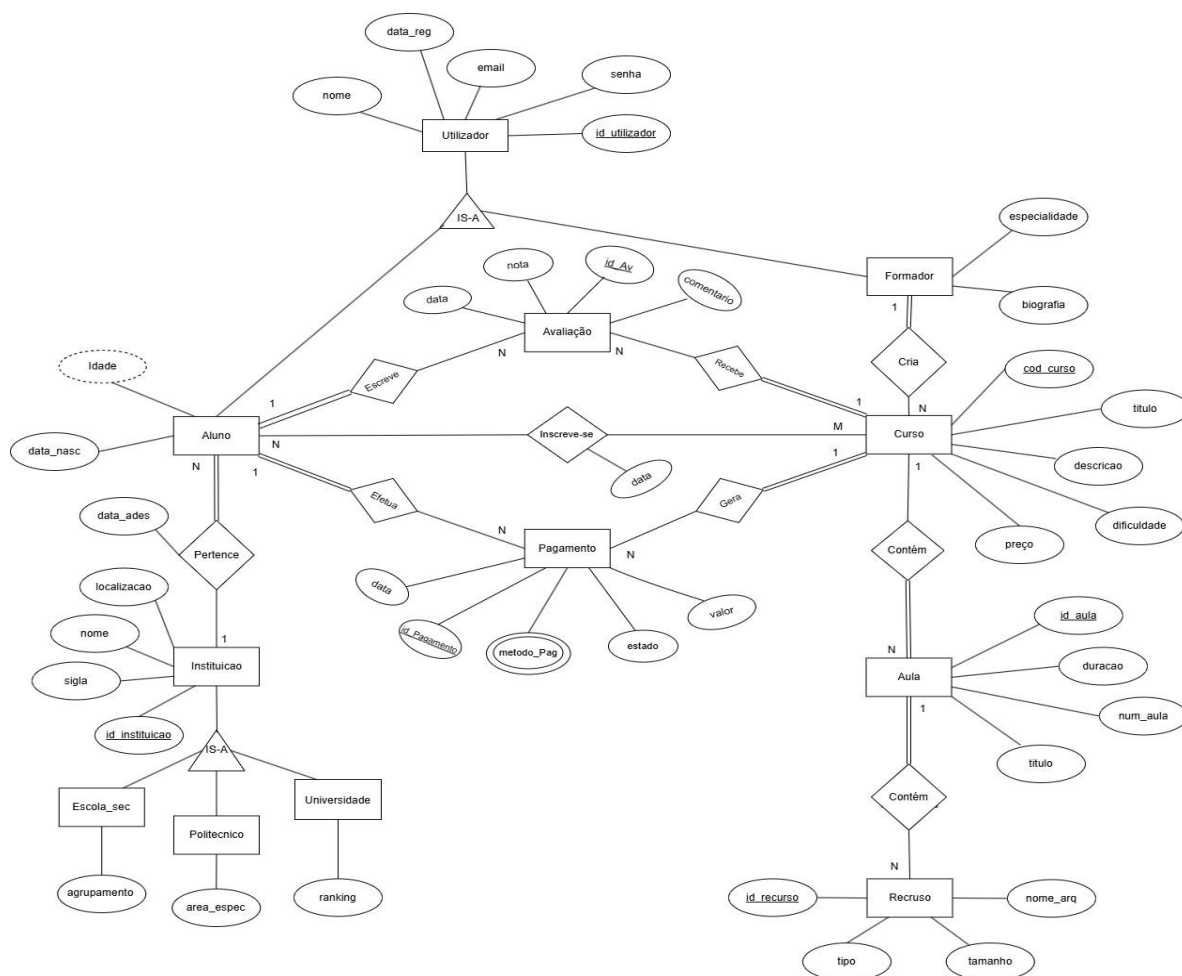


Fig.1 - Modelo Entidade-Relacionamento (DER)

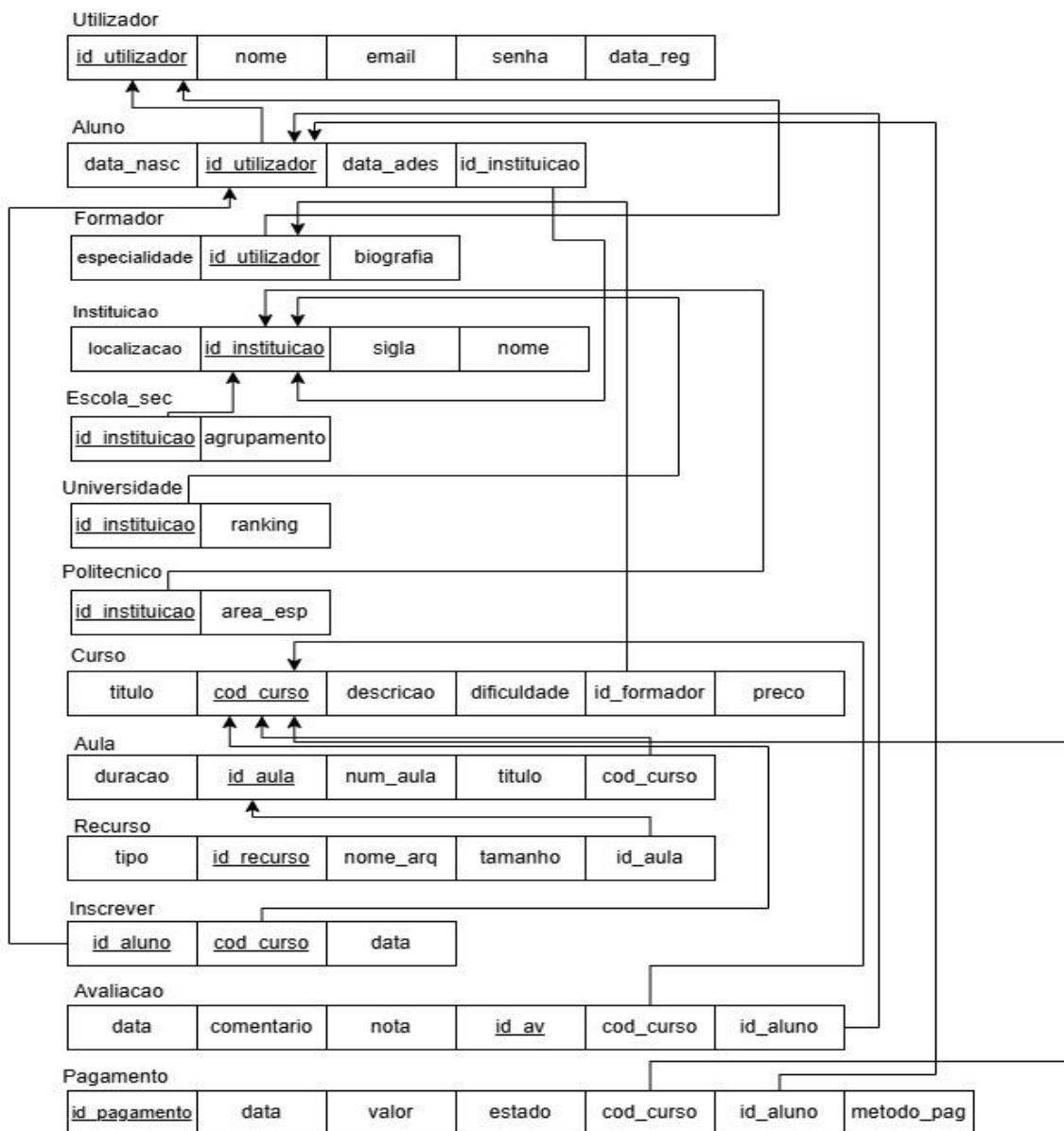


Fig.2 - Esquema Relacional (ER)

4. SQL DDL

O schema Explicacao_Online foi criado especificamente para este projeto e a estrutura da base de dados foi definida com recurso a este mesmo schema. Assim sendo, foi criado um schema chamado Explicacao_Online dentro da base de dados p1g2 que estabelece a estrutura relacional do sistema, definindo tabelas, chaves primárias (PK), chaves estrangeiras (FK) e restrições de integridade (Constraints).

Estrutura das entidades (Tabelas)

- **Utilizadores (Herança):** Existe uma tabela central Utilizador que se divide em dois subtipos: Aluno e Formador.
- **Instituições (Herança):** Tabela central Instituicao que se divide em três níveis de ensino: Escola_sec, Universidade e Politecnico.
- **Ensino e Conteúdos:** Os formadores criam Cursos. Cada curso é composto por várias Aulas, que por sua vez contêm Recursos (ficheiros).
- **Interações dos Alunos:**
 - **Inscriver:** Registo da matrícula dos alunos nos cursos.
 - **Avaliacao:** Sistema de *feedback* dos cursos com validação de notas (1 a 5).
 - **Pagamento:** Controlo financeiro com três estados possíveis (Pendente, Concluído, Cancelado).

Detalhes Técnicos (Constraints e Alterações)

- **Integridade Referencial:** Uso intensivo de Chaves Primárias (PK) e Chaves Estrangeiras (FK).
- **ON DELETE CASCADE:** Utilização de ON DELETE CASCADE para garantir que, ao eliminar um registo principal (ex: um curso), todos os registos dependentes (ex: aulas desse curso) são apagados automaticamente.
- **CHECK:** Garantia de consistência nos dados inseridos (ex: notas e estados de pagamento).
- **ALTER TABLE:** O script inclui alterações finais para corrigir as chaves estrangeiras das tabelas de Inscrição, Avaliação e Pagamento, ligando-as diretamente ao ID do Utilizador em vez do ID do Aluno.

5. SQL DML

O DML divide-se em duas partes principais: as queries geradas de forma dinamica pelos formulários da aplicação C# e os dados de teste inseridos inicialmente. Foram assim inseridos dados representativos em todas as tabelas de modo a suportar os testes das **SPs**, **UDFs** e **Triggers**.

Dados de teste iniciais que foram inseridos

- **Utilizador** -> João Miguel Silva, Maria Fernandes Costa, Ana Beatriz Pereira, Carlos Nogueira, Tiago Mendes, Sofia Lima
- **Aluno** -> João(UA),
- **Formador** -> Maria (Base de Dados), Carlos (Programação Web)
- **Instituição** -> UA, ESJE, IPC, UC
- **Escola_sec** -> ESJE – Agrupamento de Escolas José Estêvão
- **Universidade** -> UA (ranking 5), UC (ranking 6)
- **Politecnico** -> IPC – Engenharia e Gestão
- **Curso** -> Introdução ao SQL (Maria, 29.99 euros, iniciante), Dispositivos Conectados (Carlos, 49.50 euros, avançado)
- **Aula** -> 3 aulas distribuídas pelos 2 cursos (45, 60 e 90 min)
- **Recurso** -> 2 PDF e 1 ZIP distribuídos pelas aulas
- **Inscriver** -> João e Tiago no curso 1, Sofia e Ana Beatriz no curso 2
- **Avaliacao** -> 4 avaliações com notas 5, 3, 4 e 5
- **Pagamento** -> 1 pendente (Ana Beatriz 49.50 euros) e 3 concluídos (João 29.99 euros, Tiago 29.99 euros, Sofia 49.50 euros)

DML associado a cada formulário

- **Gestão de Utilizadores** (SELECT com LEFT JOIN a Aluno e Formador para detetar o tipo de utilizador numa só query, INSERT em duas tabelas numa transação (Utilizador + Aluno ou Formador), UPDATE nas duas tabelas em transação, DELETE que é controlado por impedirEliminarUtilizadorComDividas)

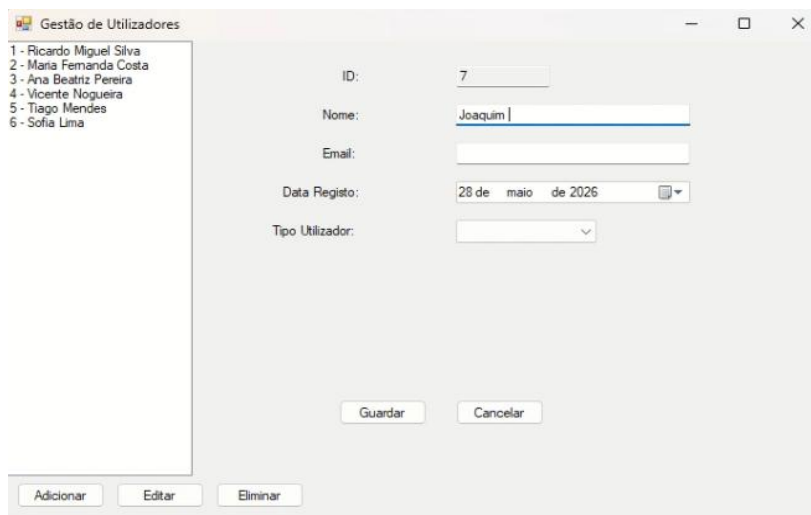


Fig.3 – Formulário de Gestão de Utilizadores

- **Gestão de Instituições** (SELECT com LEFT JOIN a Escola_sec, Universidade e Politecnico numa só query para detetar o tipo e os atributos específicos de cada instituição, INSERT em duas tabelas numa transação (Instituicao + subtipo correspondente), UPDATE nas duas tabelas em transação, DELETE controlado pelo trigger impedirEliminarInstituicaoComAlunos que impede a eliminação de instituições com alunos associados)

Fig.4 – Formulário de Gestão de Instituições

- **Catálogo de Cursos** (SELECT com chamada às UDFs mediaAvaliacaoCurso e cursosFormador ao selecionar um curso, INSERT/UPDATE/DELETE em Curso com eliminação prévia de pagamentos, filtros dinâmicos via UDF cursosMaisPopulares com ORDER BY variável (mais inscritos, melhor avaliação,..))

Fig.5 – Formulário de Catálogo de Cursos

- **Inscrições** (execução da SP registrarInscricao, execução da SP cancelarInscricao para cancelar inscrições pendentes, Trigger bloquearInscricaoPessoaADever dispara de forma automática no INSERT em Inscrever)

Inscrições

Utilizador: 7 - Joaquim Bastos (Formador)

Tipo: Formador

Curso: 3 - Métodos Probabilísticos (21.99€)

Info Curso: Grátis (formador do curso)

Método Pagamento: N/A

Deseja cancelar alguma inscrição?

Inscrições existentes:

João Dias -> Métodos Probabilísticos (28/05/2026)
Sofia Lima -> Dispositivos Conectados (16/09/2025)
Ana Beatriz Pereira -> Dispositivos Conectados (15/09/2025)
Tiago Mendes -> Introdução ao SQL - DML e DDL (06/09/2025)
Ricardo Miguel Silva -> Introdução ao SQL - DML e DDL (05/09/2025)

Fig.6 – Formulário de Inscrições

- **Pagamentos** (execução da SP processarPagamento para concluir pagamentos pendentes, chamada à UDF extratoFinanceiroAluno para mostrar o extrato financeiro completo do utilizador selecionado)

Pagamentos

5 | João Dias | Métodos Probabilísticos | 21.99€ | Pendente
 2 | Ana Beatriz Pereira | Dispositivos Conectados | 49.50€ | Concluído
 4 | Sofia Lima | Dispositivos Conectados | 49.50€ | Concluído
 3 | Tiago Mendes | Introdução ao SQL - DML e DDL | 29.9€ | Concluído
 1 | Ricardo Miguel Silva | Introdução ao SQL - DML e DDL | 29.9€ | Concluído

ID Pagamento: 5

Utilizador: João Dias

Curso: Métodos Probabilísticos

Valor: 21.99€

Estado: Pendente

Método Pag.: Cartão de Crédito

Data: 28/05/2026

Extrato de: João Dias

Fig.7 – Formulário de Pagamentos

- **Avaliações** (SELECT de utilizadores inscritos em algum curso (JOIN Inscrever), INSERT em Avaliacao – os Triggers validarCondicoesAvaliacao e impedirAutoAvaliacao disparam automaticamente e em caso de erro é apresentado ao utilizador)

Fig.7 – Formulário de Avaliações

NOTA: Para além destes formulários falta ainda aqui o inicial (formulário do menu onde vai ser depois possível escolher entre estes formulários)

6. SQL Programming

Esta secção demonstra a lógica de negócio encapsulada na base de dados através de Stored Procedures (SPs), Triggers e User-Defined Functions (UDFs).

Stored Procedures

registarInscricao — Regista a inscrição de um aluno e cria o respetivo pagamento pendente numa única transação, garantindo atomicidade. Caso o utilizador seja o formador do próprio curso, a inscrição é registada a custo zero e com pagamento automaticamente concluído.

cancelarInscricao — Permite cancelar uma inscrição, alterando o estado do pagamento associado para "Cancelado". Esta ação possui uma validação que apenas permite o cancelamento se o pagamento ainda estiver "Pendente".

processarPagamento — Transita o estado de um pagamento de "Pendente" para "Concluído", efetuando várias validações para impedir a duplicação de processamentos em pagamentos já concluídos.

Triggers

bloquearInscricaoPessoaADever — (AFTER INSERT em Inscrever) Impede que um utilizador se inscreva num novo curso se ainda possuir pagamentos pendentes noutros cursos, efetuando o ROLLBACK da transação.

validarCondicoesAvaliacao — (AFTER INSERT em Avaliacao) Garante a integridade das avaliações: um utilizador só pode avaliar um curso se estiver formalmente inscrito e se o pagamento desse curso já estiver concluído.

impedirAutoAvaliacao — (AFTER INSERT em Avaliacao) Regra de negócio que impede o formador de um curso de submeter avaliações ao seu próprio conteúdo.

gerirEliminacaoUtilizador — (INSTEAD OF DELETE em Utilizador) Gere de forma centralizada e segura a eliminação de um utilizador. Bloqueia a eliminação se existirem dívidas pendentes. Caso seja um formador, elimina os seus cursos em cascata para evitar erros de Foreign Key. Por fim, limpa o histórico de pagamentos e inscrições (se cancelados ou concluídos) antes de remover o utilizador.

cancelarInscricaoAoPagarCancelado — (AFTER UPDATE em Pagamento) Assegura a consistência dos dados ao remover automaticamente o registo da tabela Inscrever sempre que um pagamento muda o seu estado para "Cancelado".

impedirEliminarInstituicaoComAlunos — (INSTEAD OF DELETE em Instituicao) Impede a eliminação de uma instituição de ensino (ex: Universidade ou Escola) se esta ainda possuir alunos associados no sistema.

UDF (User-Defined Functions)

mediaAvaliacaoCurso — Função Escalar. Calcula e devolve a nota média das avaliações de um curso específico.

cursosFormador — Table-Valued Function. Devolve a lista de cursos criados por um formador específico, calculando automaticamente o número total de alunos inscritos em cada um deles.

cursosMaisPopulares — Table-Valued Function. Agrega todos os cursos da plataforma, calculando para cada um a sua média de avaliações (recorrendo à função *mediaAvaliacaoCurso*) e o número total de inscrições.

utilizadoresComDividasPendentes — Table-Valued Function. Lista os utilizadores com pagamentos no estado "Pendente", contabilizando o número total de pagamentos em atraso e o valor total em dívida para cada um.

extratoFinanceiroAluno — Table-Valued Function. Devolve o histórico financeiro completo de um aluno (nome do curso, data, valor, estado e método de pagamento). Utilizada na interface gráfica para gerar extratos.

Transações

As operações críticas do sistema foram implementadas com transações explícitas usando BEGIN TRAN / COMMIT TRAN / ROLLBACK TRAN dentro de blocos TRY/CATCH nas Stored Procedures, e SqlTransaction na aplicação C#.

Na base de dados, as três Stored Procedures usam transações:

- **registarInscricao** - a inserção em Inscrever e a inserção em Pagamento são feitas na mesma transação. Se a inserção do pagamento falhar por qualquer motivo depois da inscrição já ter sido criada, o ROLLBACK garante que a inscrição também é desfeita, evitando um estado inconsistente onde o utilizador estaria inscrito sem pagamento associado.
- **processarPagamento** - a atualização do estado do pagamento está numa transação com verificações prévias. Se o pagamento não existir ou já estiver concluído, o ROLLBACK é executado antes de qualquer alteração.
- **cancelarInscricao** - a atualização do pagamento para 'Cancelado' está numa transação. O trigger cancelarInscricaoAoPagarCancelado que remove a inscrição é parte da mesma transação implicitamente, garantindo que as duas operações (cancelar pagamento + remover inscrição) são sempre atómicas.

No lado da aplicação C#, todas as operações que envolvem múltiplas tabelas usam SqlTransaction:

- No FormUtilizadores, ao adicionar um utilizador, a inserção em Utilizador e a inserção em Aluno ou Formador são feitas na mesma transação — se a segunda inserção falhar, o registo em Utilizador também é revertido, nunca ficando um utilizador sem tipo definido.
- No FormInstituicoes, o mesmo padrão é aplicado — Instituicao e o subtipo (Escola_sec, Universidade ou Politecnico) são inseridos na mesma transação.

- No FormCursos, a eliminação de um curso elimina primeiro os Pagamentos associados (que não têm CASCADE) e só depois o Curso, tudo dentro da mesma transação.

7. Conclusão

O projeto permitiu aplicar de forma integrada e complementar os conceitos fundamentais de bases de dados relacionais. Desde a modelação conceptual com o DER, passando pela definição estrutural com o DDL normalizado, até à implementação de lógica de negócio complexa com Stored Procedures, UDFs e Triggers.

A decisão de implementar as regras de negócio diretamente na base de dados, em vez de as tratar apenas na aplicação C#, revelou-se a abordagem mais robusta visto que os dados mantêm-se íntegros independentemente de qual seja o cliente que os acede.

O uso de transações explícitas nas Stored Procedures e na aplicação C# garante também atomicidade nas operações multi-tabela e a adoção de queries parametrizadas em toda a aplicação previne ataques de SQL Injection. Os índices criados são também responsáveis pela otimização das queries mais frequentes dos formulários, UDFs e Triggers.

Como trabalho futuro, identificam-se as algumas melhorias possíveis tais como: implementação de hash seguro para as passwords, autenticação de utilizadores com login, entre outras.